UNDEF

FADENA
Facultad de Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos I

Unidad 2 / Semana 6

Año: 2025

Contenidos

Sistemas de proceso por lotes. Monoprogramación. Multiprogramación. Sistema de tiempo compartido. Servicios de un sistema operativo. Procesos y estados de los procesos. Modo núcleo y usuario. Llamadas al sistema.

Índice

1. Sistemas de proceso por lotes y monitores 3
1.2 JCL 5
1.3 Buffering 6
1.4 Spooling 7
2. Procesos 8
2.1 Requisitos del sistema operativo con respecto a los procesos 8
3. Monoprogramación y Multiprogramación 9
3.1 Monoprogramación 10
3.2 Multiprogramación 11
4. Multiprocesamiento y procesamiento distribuido 12
5. Sistema de tiempo compartido 12
6. Kernel del sistema operativo 13
7. Modo núcleo y usuario 14
8. Llamadas al sistema 15
9. Interrupciones **17**

1. Sistemas de proceso por lotes y monitores

Los sistemas operativos por lotes permiten procesar una gran cantidad de tareas con mínima o nula interacción entre los usuarios y los programas en ejecución. Dado el alto costo de los equipos, siempre se buscaban estrategias para optimizar el tiempo de uso de las computadoras.

Este enfoque representó una mejora significativa, ya que reducía los tiempos de preparación de los trabajos al ejecutarlos en serie. En lugar de procesar cada tarea individualmente, los sistemas agrupaban múltiples trabajos y los ejecutaban en conjunto, minimizando los períodos de inactividad y mejorando la eficiencia operativa.

Se reúnen todos los trabajos comunes para realizarlos al mismo tiempo, evitando la espera de dos o más trabajos como sucede en el procesamiento en serie.

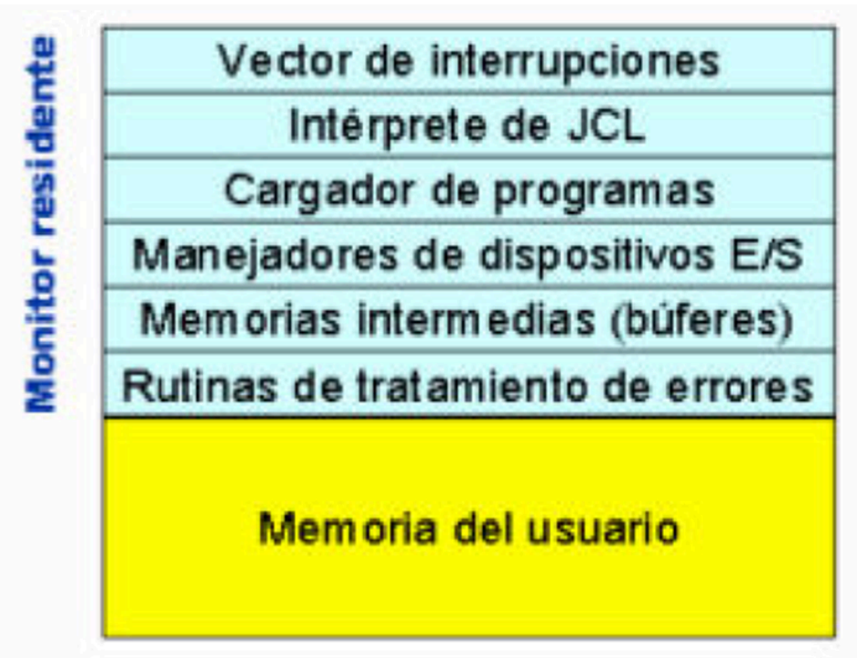
Para ello se creó un lenguaje simplificado que les permite organizar y estructurar la ejecución de esos trabajos llamado JCL (Job Control Language) que fue evolucionando y aún se sigue usando en la actualidad.

Estos sistemas son de los más tradicionales y antiguos, y fueron introducidos alrededor de 1956 para aumentar la capacidad de procesamiento de los programas.

Un sistema de lotes es aquel cuyos trabajos (jobs) son reunidos con las instrucciones necesarias para permitirles ser procesados sin intervención de un operador.

El monitor residente (o monitor), es el software del sistema que es responsable de interpretar y ejecutar las instrucciones en los trabajos por lotes.

Cuando el monitor comenzaba un trabajo, cedía el control completo de la computadora al trabajo, el cual controlaba la computadora hasta su término.



Esquema básico de la memoria de una computadora con procesamiento por lotes con monitor

residente.

Un ejemplo de varios trabajos en lote podría verse así:

```
$JOB user_spec        ; identificar al usuario
$FORTRAN              ; cargar el compilador de FORTRAN
source program cards  ; tarjetas con el programa fuente
$LOAD                 ; cargar el programa compilado
$RUN                  ; ejecutar el programa
data cards            ; tarjetas con los datos
$EOJ                  ; fin del trabajo

$JOB user_spec        ; identificar un nuevo usuario
$LOAD application     ; cargar aplicación
$RUN                  ; ejecutar la aplicación
data
$EOJ                  ; fin del trabajo
```

Por lo general se usaban cintas magnéticas o tambores para almacenar los datos intermedios y programas compilados. Las ventajas y desventajas de este tipo de sistemas son:

1. Ventajas

- reducción del tiempo ocioso del procesador entre cada trabajo
- mejora el desempeño, debido a que los trabajos inician inmediatamente después que finalice el anterior
- disminuye la carga de trabajo del operador

2. Desventajas

- tiene una depuración complicada de programas
- sin un debido esquema de protección, un trabajo del lote puede afectar a los demás trabajos pendientes
- un trabajo puede corromper al monitor, afectando también a los trabajos pendientes
- un trabajo puede entrar en un ciclo infinito (loop)

Una de las mayores carencias de los sistemas por lotes era que no existía un esquema de protección para prevenir que algún trabajo afecte adversamente a otros.

Una solución a esto fue un simple esquema de protección, donde cierta memoria (p.ej. donde reside el monitor) era restringida a los programas de usuario. Esto prevenía que los programas de usuario corrompieran al monitor.

Otra mejora fue adaptar el hardware para permitir a la computadora operar en uno de dos modos:

- uno para el monitor, que realizaba todas las tareas de E/S
- uno para los programas de usuario

La entrada/salida (E/S) podría solo ser realizada en modo de monitor, teniendo un mejor control de la ejecución de los trabajos.

Para prevenir un ciclo infinito, se agregaba un temporizador (timer) al sistema y la tarjeta $JOB fue modificada para que el tiempo máximo de ejecución de un trabajo fuera enviado al monitor. El sistema podría interrumpir el trabajo y regresar el control al monitor si se excediera este tiempo.

Algunos ejemplos de Sistemas Operativos por lotes exitosos son el SCOPE, del CDC6600, el cual está orientado a procesamiento científico pesado, y el EXEC II para el UNIVAC 1107 y 1108, orientado a procesamiento académico.

Estos grandes sistemas llamados *Mainframes* existen actualmente y si bien se usan en bancos y grandes corporaciones, pueden operar en un *modo híbrido*, combinando procesamiento *batch* e *interactivo* para maximizar la eficiencia y la disponibilidad del sistema.

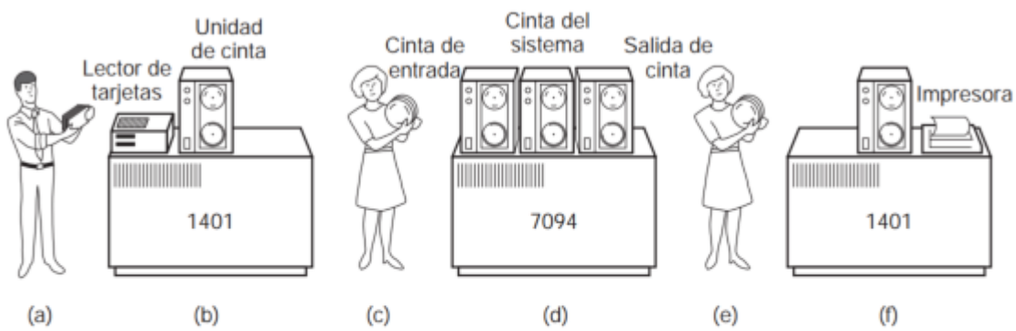


Figura 1-3. Uno de los primeros sistemas de procesamiento por lotes. a) Los programadores llevan las tarjetas a la 1401. b) La 1401 lee los lotes de trabajos y los coloca en cinta. c) El operador lleva la cinta de entrada a la 7094. d) La 7094 realiza los cálculos. e) El operador lleva la cinta de salida a la 1401. f) La 1401 imprime los resultados.

Ejemplo de un sistema batch con spooling[1]

1.2 JCL

Este lenguaje de scripting comenzó en formato de tarjetas perforadas es hoy una de las herramientas más importantes y significativas que todo aquel que trabaje con Mainframe debe dominar.

JCL es el acrónimo de *Jobs Control Language* es un lenguaje de scripting declarativo que permite a los desarrolladores de aplicaciones hablar literalmente con el z/OS, el sistema operativo para mainframes de IBM.

Por lo general, un solo JCL la fila está estructurada con 5 columnas esenciales llenas de orden fijo y las tres primeras obligatorias:

1) Doble barra (//) para el trabajo inicial.

2) Un nombre de trabajo.

3) Una instrucción de operación.

[1] Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos. 3era edición. Pearson Educación, México

4) Instrucciones de parámetros.

5) Comentarios, indicados con el símbolo /* en la línea de salida también.

Un simple JCL script que imprime una cadena de Hello World usando el programa de copia de IBM llamado IEBGENER se ve así:

```
/* Start of JCL Job
//JCLPRGR  JOB
//STEP1 EXEC PGM=IEBGENER COMMENT: EXECUTION COMMAND
//* Data to be copied
//SYSUT1   DD *
Hello, world JCL!
//* printing Hello, world JCL! string to the system output
//SYSUT2   DD SYSOUT=A
//SYSPRINT DD SYSOUT=A
//SYSIN    DD DUMMY
//
```

JOB identifica el inicio del trabajo.

EXEC se utiliza para identificar el nombre de un programa o procedimiento de unidad de ejecución a ejecutar (en nuestro caso el programa IEBGENER).

Data Definition (**DD**) se utiliza para conectar los nombres de archivo de programa utilizados por el programa indicado en la operación EXEC.

En este caso, no se usan archivos, pero usamos la salida estándar con el parámetro **SYSOUT**

El parámetro **DUMMY** significa que no se utilizarán datos de entrada para este JOB.

Finalmente, tenemos otro **"//"** al final del script con instrucciones nulas que indica el final de la ejecución del trabajo.

1.3 Buffering

El buffering consiste en almacenar temporalmente datos en la memoria antes de enviarlos a un dispositivo de salida o recibirlos de uno de entrada. Su propósito es atenuar la diferencia de velocidad entre el procesador y los periféricos.

Ejemplo:

- Cuando un programa lee datos de un disco, estos se cargan en un buffer (área de memoria) para que el procesador pueda acceder a ellos más rápidamente, evitando esperas innecesarias.

- Cuando se reproduce un video en streaming, el sistema almacena algunos segundos de contenido en un buffer para prevenir interrupciones si la conexión se ralentiza.

El buffering es esencial para mantener el rendimiento del sistema operativo y minimizar el impacto de las operaciones de E/S.

1.4 Spooling

Uno de los principales desafíos de los primeros sistemas de procesamiento por lotes era que la computadora debía leer el bloque de tarjetas perforadas antes de ejecutar un trabajo. Esto provocaba que el CPU permaneciera inactivo durante estas operaciones, que eran relativamente lentas.

Para solucionar este problema, se comenzó a utilizar cintas magnéticas, cuya lectura era mucho más rápida que la de las tarjetas perforadas. Así, los centros de cómputo adoptaron una estrategia en la que una o más computadoras de menor capacidad complementaban al mainframe principal.

Estas computadoras secundarias se encargaban de leer las tarjetas y transferir la información a cintas, permitiendo que cada cinta almacene múltiples trabajos en lote. Posteriormente, la cinta se cargaba en la computadora principal, que ejecutaba los trabajos de manera eficiente. Una vez procesados, los resultados podían guardarse en otra cinta, que luego era utilizada por una computadora de menor capacidad para generar las salidas necesarias, como impresiones.

Una evolución lógica dentro de los sistemas por lotes fue la introducción de temporizadores, que limitaban el tiempo de ejecución de los trabajos antes de interrumpirlos para permitir operaciones de entrada/salida (E/S). Esto habilitaba la ejecución en paralelo de procesos, permitiendo que la computadora leyera tarjetas y almacene la información en cintas, tambores o discos, mientras seguía procesando programas de usuario.

Este procedimiento se conoce como **SPOOL** o SPOOLing (Simultaneous Peripheral Operation On-Line), y fue clave en la evolución de los sistemas de procesamiento por lotes.

El SPOOL va más allá del buffering, ya que no solo almacena los datos temporalmente, sino que los gestiona de manera autónoma en colas de trabajo, permitiendo que el procesador continúe ejecutando otras tareas sin interrupciones.

Los sistemas por lotes con spooling fueron los primeros y más simples sistemas de *multiprogramación*, ya que permitían que los trabajos se procesen de manera más eficiente. Una de sus principales ventajas era que la salida de un trabajo estaba disponible inmediatamente después de su finalización, en lugar de esperar a que todos los trabajos del lote se completaran.

Ejemplo:

Además del caso de la lectura de tarjetas en unidades de cinta ya mencionado, también se usan en la impresión de documentos, cuando varios usuarios envían trabajos a la misma impresora, el sistema almacena las solicitudes en un spool (cola de impresión) y las procesa en orden, evitando que la CPU se detenga esperando que cada impresión se complete.

Mientras que el buffering optimiza la transferencia de datos entre el CPU y los periféricos, el spooling gestiona la ejecución de múltiples trabajos de manera más organizada, liberando el procesador para realizar otras tareas.

Ambas técnicas mejoran la eficiencia del sistema operativo y optimizan la gestión de recursos y fueron de gran ayuda para optimizar los sistemas de procesamiento por lotes aumentando su eficiencia.

2. Procesos

En los sistemas operativos, un **proceso** es la instancia de un programa en ejecución. Representa la unidad fundamental de trabajo dentro del sistema, donde cada proceso incluye **código ejecutable**, **datos**, los recursos que se le asignaron (como memoria y archivos abiertos) y su propio **contexto de ejecución**.

El contexto de ejecución incluye toda la información que el sistema operativo necesita para administrar el proceso y que el procesador necesita para ejecutarlo correctamente. Por lo tanto, el contexto incluye los contenidos de varios registros del procesador, como el contador de programa y los registros de datos. También incluye información útil para el SO, como la prioridad del proceso con respecto a su ejecución y si el proceso está esperando la terminación de un suceso de E/S.

En resumen, cuando ejecutamos una aplicación, el sistema operativo crea un proceso. Por ejemplo, cuando ejecutamos un procesador de textos, planilla de cálculo, navegador web, gestor de tareas, antivirus o cualquier otra que abramos para trabajar, el sistema operativo crea procesos para cada una de ellas y le asigna un identificador único. Además, cada proceso incluirá los elementos antes indicados.

Por otro lado, cada proceso puede estar en diferentes **estados** como **activo**, **suspendido** o **finalizado**, dependiendo de su uso y administración por parte del sistema operativo.

2.1 Requisitos del sistema operativo con respecto a los procesos

Los sistemas operativos modernos están construidos en torno al concepto de proceso. Por tal motivo, la mayoría de los requisitos que debe cumplir un sistema operativo están expresados haciendo referencia a los procesos:

- El sistema operativo debe intercalar la ejecución de múltiples procesos para optimizar al máximo la utilización del procesador ofreciendo un tiempo de respuesta razonable.
- El sistema operativo debe asignar los recursos a los procesos teniendo en cuenta ciertas políticas, como por ejemplo la prioridad de ejecución de algunas aplicaciones.
- El sistema operativo tendría que dar soporte a la comunicación entre procesos.

3. Monoprogramación y Multiprogramación

Se refiere a la capacidad de un sistema operativo de ejecutar múltiples procesos, también llamados tareas, dándole al usuario o los usuarios la impresión de que las aplicaciones son ejecutados simultáneamente y sin interferir los unos con los otros.

Cada proceso al ejecutarse, es como si fuera el único dentro del sistema que se está ejecutando y con acceso a todos los recursos y servicios que nos brinda el sistema operativo.

Los procesos que se ejecutan de forma concurrente (al mismo tiempo) pueden representar:

- Programas diferentes
- Partes diferentes de un mismo programa
- Diferentes instancias de un mismo programa (cada instancia es una copia en ejecución del mismo programa)

El número de procesos que pueden ejecutarse en un sistema depende de varios factores entre los que se incluyen:

- El tamaño de la memoria
- La velocidad del procesador central (CPU)
- El tamaño de los programas
- La tecnología del procesador

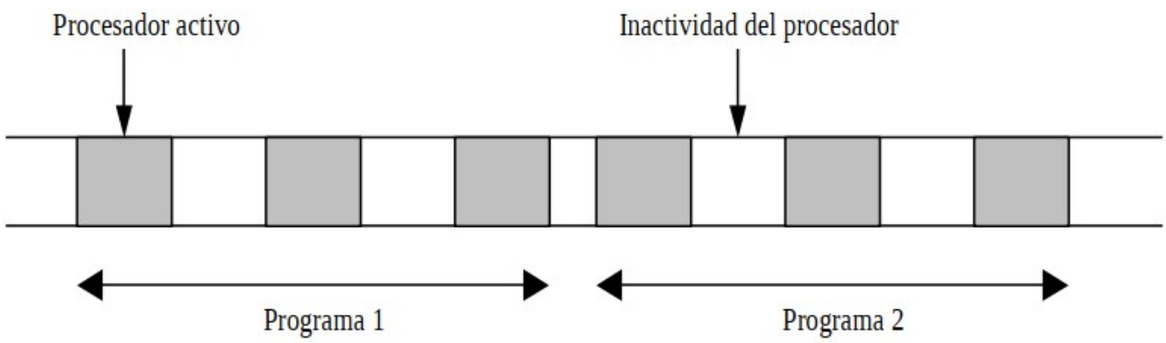
Una característica importante del sistema operativo para poder soportar la multitarea es que cada proceso esté **protegido** del resto de los procesos. Protegido quiere decir que se ejecuta en un espacio físico y lógico que garantiza su ejecución y que no es alcanzado accidentalmente por ningún otro proceso, no quiere decir aislado completamente (existen formas de comunicar un proceso con otro si es necesario).

El procesador o procesadores son demandados continuamente en el sistema operativo por numerosos procesos que compiten por sus servicios. Pero como el procesador solo puede ejecutar un proceso al mismo tiempo, la multiprogramación es acompañada de una técnica de ranuras de tiempo (time slices) que permiten alternar el uso del procesador entre varios procesos a una alta velocidad.

Desde el punto de vista de la ejecución de procesos, los sistemas operativos se dividen en dos grandes grupos:

3.1 Monoprogramación

Existe un único proceso, el procesador ejecuta las instrucciones hasta que alcanza una de tipo E/S. Entonces debe esperar que la instrucción de E/S concluya antes de continuar. Esto implica que el procesador tenga tiempos que no trabaja, desperdiciando su velocidad de procesamiento mientras un dispositivo de E/S, por ejemplo una impresora imprime un documento. El imprimir o guardar datos en un disco son tareas muy lentas comparadas con la velocidad de procesamiento de los procesadores. Por ejemplo, fue el MS-DOS.



Esquema de funcionamiento del procesador en un sistema operativo

En el esquema, se observa cómo el procesador alterna entre estar **activo** (representado en gris) e **inactivo** (representado en blanco) mientras ejecuta diferentes programas.
En la monoprogramación, solo se puede ejecutar un programa a la vez. Esto significa que el procesador dedica su tiempo a un solo programa hasta que este se completa y debe esperar por algún recurso, por ejemplo, imprimir un documento. Esto implica que el procesador tendrá tiempos ociosos. Una vez finalizado o en espera, el procesador puede pasar a ejecutar otro programa.

Procesador activo o inactivo

Con respecto, al procesador activo, indica que durante los períodos en que el bloque está en gris, el procesador está ejecutando instrucciones del programa y durante los períodos en que el bloque está en blanco, el procesador está inactivo, posiblemente esperando la finalización de una operación de E/S o que el programa termine su ejecución.

Programa 1 y Programa 2

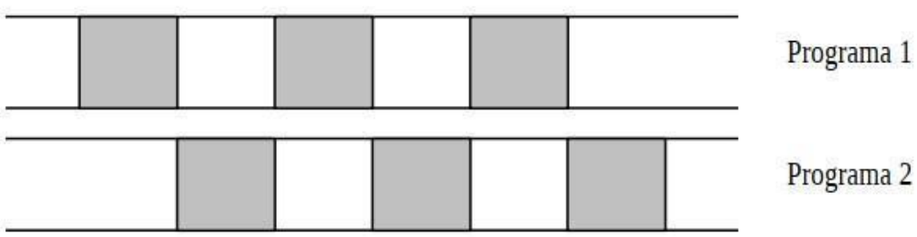
En el esquema se muestra la ejecución secuencial de dos programas. Primero, el Programa 1 se ejecuta con periodos de actividad e inactividad del procesador.

Luego, una vez finalizado el Programa 1, se ejecuta el Programa 2, también con sus propios periodos de actividad e inactividad.

Como conclusión, podemos decir que la monoprogramación es un enfoque simple pero ineficiente, ya que el procesador puede permanecer inactivo durante largos periodos mientras espera, por ejemplo, la entrada o salida de datos. Los sistemas operativos modernos utilizan técnicas más avanzadas como la multiprogramación y la multitarea para optimizar el uso del procesador.

3.2 Multiprogramación

En un sistema con un solo procesador se pueden ejecutar simultáneamente **múltiples procesos**. Esto se refiere a la gestión de varios procesos en un sistema monoprocesador. Además, el usuario percibe que todos los procesos se están ejecutando en paralelo. Prácticamente todos los sistemas operativos actuales como Windows, Mac OS o UNIX/Linux permiten la multiprogramación.



Esquema de funcionamiento del procesador en un sistema operativo multiprogramado

El esquema muestra un ejemplo de multiprogramación donde se representan dos programas, Programa 1 y Programa 2, ejecutándose de forma concurrente.

El programa 1, se ejecuta en tres bloques de tiempo, representados por los rectángulos grises. El programa 2, se ejecuta en tres bloques de tiempo, también representados por rectángulos grises.

Los espacios en blanco entre los bloques de tiempo representan los momentos en que el programa no está en ejecución, posiblemente esperando una operación de E/S o cediendo el procesador a otro programa.

La multiprogramación mejora la utilización de la CPU al reducir el tiempo de inactividad. En un sistema monoprogramado, si un proceso se bloquea esperando una operación de E/S, la CPU permanece inactiva. **Con la multiprogramación, el sistema operativo puede cambiar rápidamente entre procesos, asegurando que la CPU siempre tenga trabajo que realizar.**

4. Multiprocesamiento y procesamiento distribuido

- **Multiprocesamiento**: consiste en la gestión de varios procesos en un **sistema multiprocesador**. Se define generalmente el multiprocesamiento como el uso, en una misma computadora, de varios procesadores operando simultáneamente sobre programas o partes de programas diferentes. Se relaciona directamente con el concepto de multiprocesador, que se define como un sistema informático en el que varias CPU funcionan en paralelo, compartiendo un mismo conjunto de memorias y unidades periféricas.

- **Procesamiento distribuido**: consiste en la gestión de varios procesos, ejecutándose en sistemas de computadoras múltiples y distribuidas. Para manejar este tipo de procesamiento en las aplicaciones la tendencia actual es la arquitectura **cliente-servidor**. Una aplicación muy común del procesamiento distribuido es en las bases de datos, donde el procesamiento distribuido podría realizar la entrada/salida, la selección y la validación de los datos en una computadora, y luego crear un reporte basado en esos datos o una consulta en otra computadora.

5. Sistema de tiempo compartido

Durante la tercera generación de sistemas operativos, aunque se usaban para cálculos científicos y procesamiento de datos comerciales, seguían funcionando por lotes, lo que generaba largos tiempos de espera para los programadores.

Esto llevó al desarrollo de los sistemas de tiempo compartido (timesharing), un sistema que permitía a múltiples usuarios interactuar con una computadora en línea, asignando la CPU y otros recursos según la demanda.

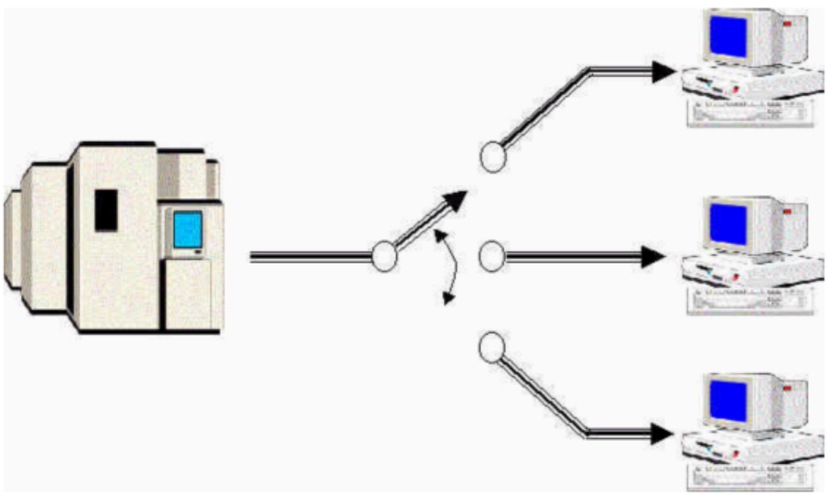
El primer sistema de tiempo compartido fue *CTSS*, desarrollado en el *MIT*, pero la tecnología no se popularizó hasta que la protección de hardware se hizo común. Posteriormente, el MIT, Bell Labs y General Electric desarrollaron MULTICS, un sistema ambicioso diseñado para dar servicio a cientos de usuarios simultáneamente. Aunque no tuvo gran éxito comercial, introdujo ideas fundamentales que influenciaron a sistemas posteriores.

Tras la salida de Bell Labs del proyecto MULTICS, Ken Thompson creó una versión simplificada en una minicomputadora PDP-7, lo que dio origen a UNIX.

UNIX se expandió en distintas versiones como System V de AT&T y BSD de Berkeley, lo que llevó a la creación del estándar POSIX para garantizar compatibilidad.

Finalmente, MINIX, un clon de UNIX desarrollado con fines educativos, evolucionó en MINIX 3, un sistema modular enfocado en la alta confiabilidad.

Aunque MULTICS no tuvo el éxito esperado, influyó enormemente en el desarrollo de sistemas operativos modernos y su legado persiste en el diseño de servidores y sistemas centralizados.



Sistema de tiempo compartido[2]

6. Kernel del sistema operativo

El **kernel** es el núcleo o componente central de un sistema operativo. Su función principal es **administrar los principales recursos de la computadora,** para que las aplicaciones puedan ejecutarse de manera eficiente y segura.

En esencia, el kernel se encarga de la **gestión de la memoria**, asignando y liberando espacio para los procesos, y de la **gestión del tiempo de la CPU**, decidiendo qué proceso se ejecuta en cada momento. Es un componente **fundamental e indispensable** de cualquier sistema operativo.

El kernel actúa como una **capa de abstracción** entre las aplicaciones de usuario y el hardware subyacente. Facilita esta interacción a través de dos mecanismos principales: la **comunicación entre procesos (IPC)**, que permite que diferentes programas intercambien información, y las **llamadas al sistema**, que son solicitudes que las aplicaciones hacen al kernel para acceder a los recursos del hardware o realizar operaciones privilegiadas.

[2] Diseño y Simulación de Sistemas Operativos, E.Hernández

7. Modo núcleo y usuario

Es necesario distinguir entre los **modos de ejecución del procesador**, normalmente asociados con el **sistema operativo** (**modo núcleo**) y con los **programas de usuario** (**modo usuario**).

Imaginemos que un usuario pudiera hacer cambios en tiempo real de las instrucciones que rigen el funcionamiento del sistema operativo y que interactúan con el hardware, que pudiera cambiar el controlador de interrupciones de reloj, la forma en la que se produce la comunicación entre un dispositivo E/S. Si esto sucediera, se podría hacer un uso indebido del sistema operativo y del hardware, produciendo comportamientos inesperados y pudiendo corromper su funcionamiento. Por eso, es necesario que determinadas instrucciones del sistema operativo sólo puedan ser ejecutadas en modos privilegiados. Éstas serían lectura y modificación de los registros de control, instrucciones primitivas de E/S, instrucciones relacionadas con la memoria, instrucciones sobre señales, planificación, interrupciones, controladores de dispositivos, etc.

El modo más privilegiado de ejecución se conoce como modo núcleo o modo kernel. El núcleo de un sistema operativo es la parte que engloba las funciones y servicios más importantes del mismo. Es una parte relativamente pequeña del sistema, pero es la más utilizada, por ello suele residir en memoria de forma continua.

El motivo de que existan distintos modos de ejecución no es otro que proteger al sistema operativo y programas de usuario de alguna de ellas que acceda a funciones propias del sistema operativo en perjuicio del mismo y de otras aplicaciones de usuario que se estén ejecutando. En modo núcleo, el software tiene control completo del procesador y de sus instrucciones, registros y memoria.

Como solución a estos problemas, las aplicaciones de usuario se ejecutan en modo usuario y cuando necesitan realizar funciones que no tienen permitidas, realizan una **llamada al sistema** y como el sistema operativo tiene acceso a todas las instrucciones del procesador, realiza las mismas cambiando previamente a modo núcleo y cuando finaliza la ejecución del servicio, el modo se fija de nuevo a modo usuario.

Modo protegido, ó modo de núcleo: permite el acceso directamente al hardware y hacer operaciones de bajo nivel. **Modo de usuario**: no se permite acceder directamente al hardware, sólo se permite utilizar las llamadas al sistema, que son funciones proporcionadas por el SO para acceder al hardware.

En resumen....

Todas las tareas que necesitan realizar las aplicaciones no pueden realizarse sin la ayuda del sistema operativo. En realidad, necesitamos siempre del sistema operativo, ya que para poder ejecutar un proceso necesitamos del sistema operativo, y sin un proceso ejecutándose no podemos hacer nada. Ahora, supongamos que tenemos a un proceso ejecutándose en el sistema, qué es lo que éste puede hacer sin emplear llamadas al sistema, la respuesta será muy poco. Este podrá ejecutar todas las instrucciones que la arquitectura del procesador permite que se ejecuten en modo usuario pero nada más. Si tenemos en cuenta que toda comunicación entre el procesador y el mundo exterior se hace mediante dispositivos periféricos (teclado, monitor, placa de red, etc.) y que es el sistema operativo el encargado de administrarlos, y por lo tanto, el único que puede comunicarse con ellos, nos daremos cuenta hasta qué punto somos dependientes del sistema operativo para poder realizar cualquier tipo de tarea. Con lo cual, todo lo que un usuario de un sistema operativo puede hacer se encuentra acotado por las instrucciones que la arquitectura del sistema en que estamos trabajando nos permite ejecutar en modo usuario, más las tareas que pueden ser realizadas por medio de llamadas al sistema.

A grandes rasgos las diferencias entre **modo kernel** (Protected Mode) y **modo usuario** son las siguientes:

- En **modo kernel** se encuentran disponibles todas las instrucciones y funcionalidades que la arquitectura del procesador es capaz de brindar, sin ningún tipo de restricciones. Es en este modo en el cual se ejecuta el kernel (núcleo) del sistema operativo. Por lo general, es el kernel el único que se ejecuta en este modo.
- En **modo usuario** se encuentran disponibles solamente un subconjunto de las instrucciones y funcionalidades que la arquitectura del procesador ofrece . En este modo se ejecutan los procesos de los usuarios de sistema.

8. Llamadas al sistema

Las llamadas al sistema son el mecanismo por medio del cual las aplicaciones solicitan servicios al sistema operativo. Cuando un programa de usuario necesita realizar alguna operación con algún objeto de la máquina (por ejemplo, acceder al disco, impresora, pantalla, entre muchos otros) no puede hacerlo directamente y lo hace a través de una **llamada al sistema**; invoca al sistema operativo y este realiza la operación en su lugar.

¿Por qué son necesarias?

Los sistemas operativos modernos están diseñados con una separación clara entre el **modo usuario** (donde se ejecutan las aplicaciones con privilegios limitados) y el **modo kernel** (donde el SO tiene control total sobre el hardware y los recursos del sistema). Esta división es fundamental para la estabilidad y seguridad del sistema, ya que impide que las aplicaciones interfieran directamente con el hardware u otros procesos de manera inadecuada.

Cuando una aplicación necesita realizar una operación que excede sus permisos, como leer o escribir un archivo, crear un nuevo proceso, gestionar la memoria o comunicarse con un dispositivo de hardware, debe recurrir al sistema operativo a través del uso de las llamadas al sistema.

Tipos Comunes de Llamadas al Sistema

Los sistemas operativos ofrecen una amplia variedad de llamadas al sistema para gestionar diferentes aspectos del sistema. Algunas categorías comunes incluyen:

- **Gestión de Procesos:** Creación (`fork`, `CreateProcess`), terminación (`exit`, `TerminateProcess`), carga y ejecución (`exec`), espera de finalización (`wait`).
- **Gestión de Archivos:** Apertura (`open`), cierre (`close`), lectura (`read`), escritura (`write`), obtención de atributos de archivo.
- **Gestión de Dispositivos:** Solicitud y liberación de dispositivos, lectura y escritura en dispositivos.
- **Gestión de Memoria:** Asignación y liberación de memoria (`brk`, `mmap`,`malloc`,`free`).
- **Comunicación entre Procesos (IPC):** Creación y gestión de tuberías (pipes), semáforos, memoria compartida, envío y recepción de mensajes.
- **Información del Sistema:** Obtención de la hora, fecha, atributos del sistema.

Las llamadas al sistema son indispensables para el funcionamiento de cualquier sistema operativo moderno. Proporcionan una capa de abstracción esencial que:

- **Protege el sistema:** Al centralizar las operaciones privilegiadas a través del kernel, se previene que las aplicaciones corrompan el sistema o accedan a recursos no autorizados.
- **Gestiona los recursos:** Permiten al SO administrar de manera eficiente y ordenada los recursos compartidos como la CPU, la memoria y los dispositivos de E/S.
- **Simplifica el desarrollo de aplicaciones:** Ofrecen una interfaz bien definida y estandarizada para que los desarrolladores interactúen con el SO, sin necesidad de conocer los detalles internos del hardware.
- **Facilita la portabilidad:** Aunque las implementaciones exactas de las llamadas al sistema pueden variar entre diferentes SO, el uso de estándares como POSIX ayudan a que las aplicaciones sean más portables entre sistemas compatibles.

9. Interrupciones

Casi todos las computadoras tienen un mecanismo mediante el cual otros módulos (E/S, memoria) pueden interrumpir la ejecución normal del procesador, a este mecanismo se lo denomina interrupciones. **Los sistemas operativos modernos están controlados mediante interrupciones**.

Las **interrupciones** permiten mejorar la eficiencia del procesamiento debido a que la mayoría de los dispositivos externos son mucho más lentos que el procesador y como consecuencia el procesador tendría un tiempo ocioso, por ejemplo mientras espera que la impresora se ponga al corriente. La duración de la pausa supone cientos e incluso miles de ciclos de instrucción donde el procesador no ejecuta instrucciones. Con las interrupciones, el procesador se puede dedicar a la ejecución de otras instrucciones mientras una operación de E/S está en proceso. Cuando el dispositivo de E/S esté disponible, es decir, cuando esté preparado para aceptar más datos del procesador, el módulo de E/S de dicho dispositivo enviará una señal de solicitud de interrupción al procesador. El procesador responde suspendiendo la operación del programa en curso y saltando a un programa que da servicio al dispositivo de E/S en particular, conocido como rutina de tratamiento de interrupciones (Interrupt handler), reanudando la ejecución original después de haber atendido al dispositivo.

Desde el punto de vista del programa de usuario una interrupción solamente plantea una suspensión de la secuencia normal de ejecución. Cuando el tratamiento de la interrupción termina, la ejecución continúa. El programa no tiene que disponer de ningún código especial para dar lugar a las interrupciones; el procesador y el sistema operativo son los responsables de suspender el programa de usuario y de reanudarlo después en el mismo punto. Para dar lugar a las interrupciones, se añade un ciclo de interrupción al ciclo de instrucción. En el ciclo de interrupción el procesador comprueba si ha ocurrido alguna interrupción, lo que se indicará con la presencia de alguna señal de interrupción. Si no hay interrupciones pendientes, el procesador continúa con el ciclo de lectura y trae la próxima instrucción del programa en curso. Si hay una interrupción pendiente, el procesador suspende la ejecución del programa en curso y ejecuta una rutina de tratamiento de interrupción. Esta rutina, generalmente, forma parte del sistema operativo y determina la naturaleza de la interrupción realizando cuantas acciones sean necesarias.

Un ejemplo práctico

Un programa en ejecución consta de una secuencia de instrucciones en las que se encuentran intercaladas llamadas a imprimir. Éstas son en realidad llamadas a un programa de E/S que es una utilidad del sistema que realizará la operación concreta de E/S y consta de:

1. Secuencia de instrucciones de **preparación** para la operación concreta de E/S. Puede incluir copia de datos, etc. Luego viene la **orden** concreta de E/S. Sin el uso de interrupciones el programa deberá esperar a que el dispositivo de E/S (en este caso la impresora) termine la función pedida.
2. Otra secuencia de instrucciones que permiten completar la operación de E/S. Puede incluir la activación de un código de condición que indique el éxito o fracaso de la operación.

Dado que la operación de E/S puede tardar un tiempo considerable debido al dispositivo impresor, el programa de E/S puede quedar detenido esperando a que se complete la operación. Entonces el programa de usuario se detendrá por un tiempo considerable en cada llamada a impresión. Observe la *figura 1*, donde el programa de usuario se encuentra indicado con rectángulos rojos que simbolizan el conjunto de instrucciones que se encuentran intercaladas por llamadas a imprimir. Por otra parte, el rectángulo "a" simboliza la secuencia de instrucciones de preparación para la operación de E/S y el rectángulo "b" las instrucciones que permiten completar la operación de E/S.

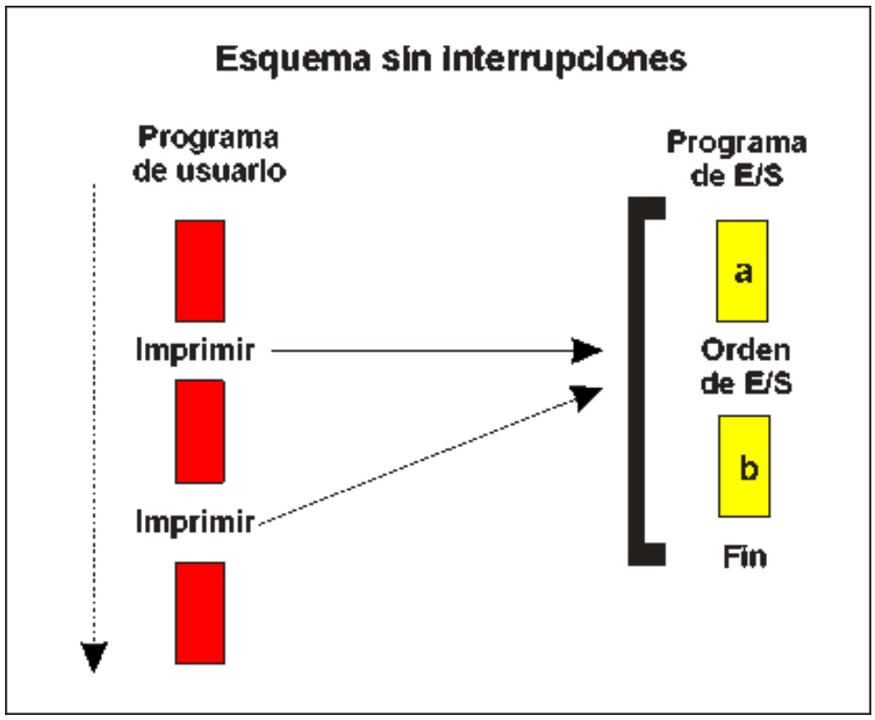


Figura 1

Con las interrupciones, el procesador se puede dedicar a la ejecución de otras instrucciones mientras una operación de E/S está en proceso. Observe la *figura 2*, cuando el programa de usuario realiza la llamada a Imprimir, se invoca al programa de E/S que en este caso solamente tiene las instrucciones de preparación y la orden de E/S. Después de ejecutarse estas pocas instrucciones, se devuelve el control al programa de usuario. Mientras tanto el dispositivo externo estará ocupado recibiendo datos de la memoria de la computadora e imprimiéndolos. Esta operación se realiza concurrentemente con la ejecución de las instrucciones del programa de usuario.

Cuando el dispositivo externo esté disponible, es decir, cuando esté preparado para recibir más datos desde el procesador, el módulo de E/S de dicho dispositivo enviará una señal de **solicitud de interrupción** al procesador. Entonces el procesador suspenderá la operación del programa en curso y comenzará la ejecución de un programa que dará servicio al dispositivo de E/S en particular. Dicho programa se denomina **rutina de tratamiento de interrupción**. Por último, el procesador reanudará la ejecución del programa original una vez que haya sido atendido el dispositivo de E/S. En la figura 2, las interrupciones están indicadas con un rombo.

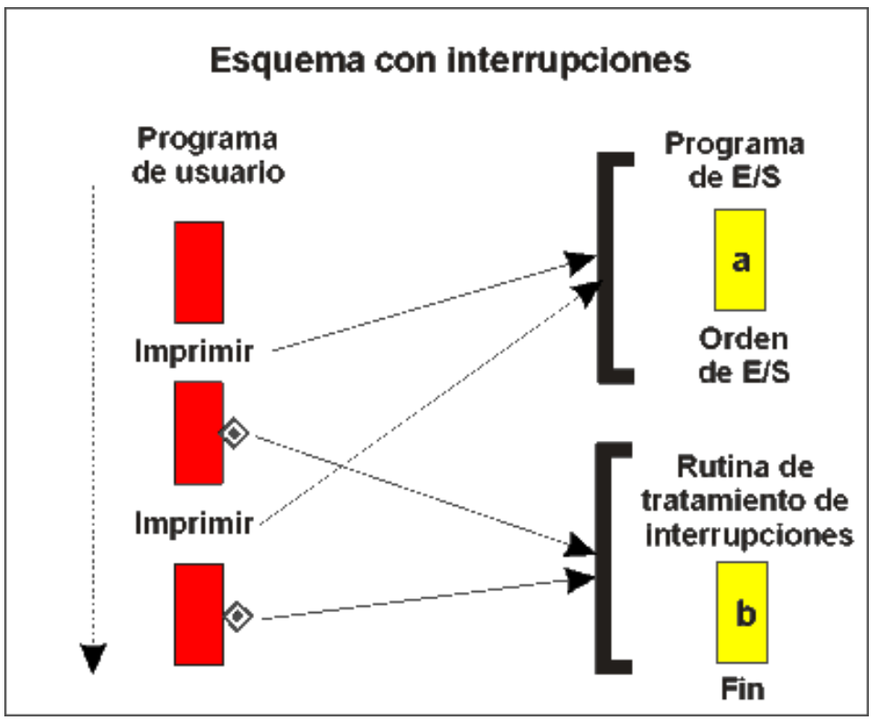


Figura 2

La siguiente tabla[3] resume brevemente las clases de interrupciones que pueden producirse:

[3] Basado en Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, quinta edición, pág. 17.

De programa (se las conoce también como Excepciones)	Producidas por alguna condición que se genera como resultado de la ejecución de una instrucción, normalmente por errores del programa, como por ejemplo: Desbordamiento aritmético. División por cero. Referencia a una dirección de memoria fuera del espacio permitido al usuario. Ejecución de una instrucción ilegal de la máquina.
De E/S	Producidas por un controlador de E/S, indicando que una operación ha terminado normalmente o para indicar alguna condición de error.
De reloj	Producidas por el reloj interno del procesador. Permiten al sistema operativo realizar determinadas funciones con cierta regularidad.
Por falla de hardware	Producidas por fallas como por ejemplo un corte de corriente o un error en la paridad de la memoria.